Comparaison et interprétation de deux méthodes d'orthogonalisation de Schmidt

Chen Nuo (20124686)
Li Yicong (20124719)

UTSEUS

Avril, 2022

MT23-Algèbre linéaire

Abstract

Au cour du *chapitre 5 : Espaces euclidiens* sur l'algèbre linéaire, on a pris la méthode originale de l'orthogonalisation de Schmidt. Cependant si on implémente le procédé en créant une fonction dans Matlab, on trouver qu'il y a une erreur importante partout la dimension de la matrice est nombreux. Donc, on propose une version modifiée qui est théoriquement équivalent mais calcule des résultats différents avec la version originale dans un ordinateur. On compare et interpréte les résultats obtenus par les deux algorithmes.

Table des matières

1 Version originale de l'orthogonalisation **2**

1.1 L'introduction d' orthogonalisation de Schmidt originale 2

1.2 Explication de l'erreur dans cette méthode 2

2 Version modifiée de l'orthogonalisation **3**

2.1 L'introduction d' orthogonalisation de Schmidt modifiée 3

2.2 Explication de cette méthode 4

3 application **4**

3.1 tester les deux procédures sur les vecteur colonnes des matrices suivantes . . 4

3.2 Comparer et interpréter les résultats obtenus par les deux algorithmes 5

4 Le lien entre l'erreur et le nombre de conditions **6**

5 annexe **8**

5.1 Schmidt_orthogonalization_Original.m 8

5.2 Schmidt_orthogonalization_Modified.m 9

5.3 erreur_moyenne.m 9

1 Version originale de l'orthogonalisation

1.1 L'introduction d' orthogonalisation de Schmidt originale

Afin d'obtenir la base orthonormée de l'espace euclidien, on utilise souvent l'orthogonalisation de Schmidt.

Soit E un espace euclidien muni du produit scalaire $<,>$.

Soit

$$V = \{v_1, \ldots, v_n\}$$

une famille libre de E, le procédé d'orthonormalisation de Schmidt consiste à construire une famille orthonormée

$$\varepsilon = \{e_1, \ldots, e_n\}$$

telle que

$$Vect < v_1, \ldots, v_j >= Vect < e_1, \ldots, e_j >, \forall j \leq n.$$

On note

$$proj_u w = \frac{< u, m >}{< u, u >} u$$

qui signifie la projection du vecteur w sur le vecteur u.

Pour k=1, il suffit de normaliser le vecteur :

$$u_1 = v_1, \qquad e_1 = \frac{u_1}{\| u_1 \|}$$

Pour k=2, 3, ..., n :

$$\begin{aligned}
&u_2 = v_2 - proj_{u_1} v_2, \qquad e_2 = \tfrac{u_2}{\| u_2 \|} \\
&u_3 = v_3 - proj_{u_1} v_3 - proj_{u_2} v_3, \qquad e_3 = \tfrac{u_3}{\| u_3 \|} \\
&\vdots \\
&u_k = v_k - \sum_{j=1}^{k-1} proj_{u_j} v_k, \qquad e_k = \tfrac{u_k}{\| u_k \|}
\end{aligned}$$

1.2 Explication de l'erreur dans cette méthode

La version originale de l'orthogonalisation est comparable à la construction avec des blocs. On prend le premier vecteur comme référence et calcule le second vecteur perpendiculaire à cette

référence. Puis on prend les deux vecteurs comme référence et calcule le troisième vecteur, et ainsi de suite.Le calcule du nième vecteur nécessite le résultat nth-1. Par conséquent, l'erreur d'un vecteur affectera la précision de chaque vecteur après celui-ci.

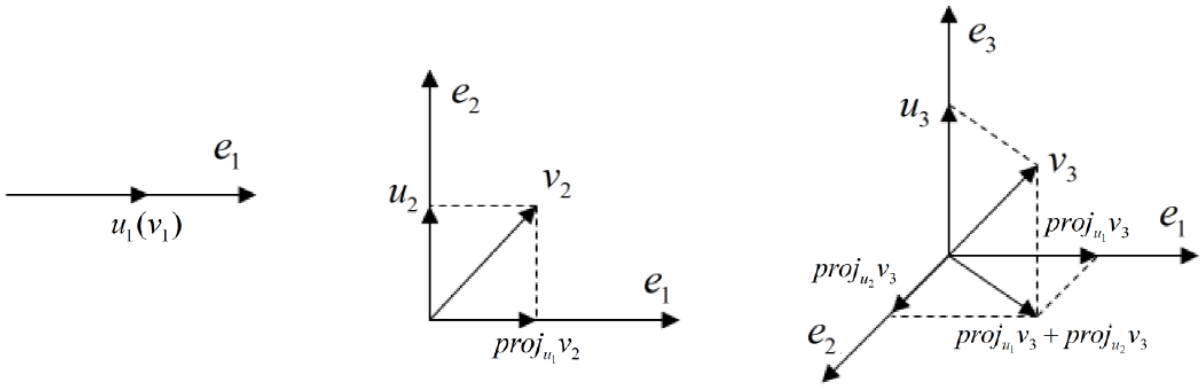


FIGURE 1 – *Illustration de l'orthogonalisation de Schmidt lorsque k=1, 2, 3*

De plus, la façon dont les ordinateurs sont calculés peut produire des résidus, en particulier lors de la soustraction de deux nombres proches l'un de l'autre. Les erreurs sont amplifiées comme une boule de neige au fur et à mesure que le calcul se poursuit

2 Version modifiée de l'orthogonalisation

2.1 L'introduction d' orthogonalisation de Schmidt modifiée

Visant le phénomène d'élargissement constant de la méthode traditionnelle, nous concevons une nouvelle méthode pour modifier le algorithme d'orthogonalisation de Schmidt.

Pour k=1, il n'y a pas de différence entre les deux méthodes :

$$u_1 = v_1, \qquad e_1 = \frac{u_1}{\| u_1 \|}$$

Pour k=2, 3, ...,n, au lieu de calculer les vecteur u_k par

$$u_k = v_k - \sum_{j=1}^{k-1} proj_{u_k} v_k$$

On procède comme suit :

$$\begin{aligned}
u_k^{(1)} &= v_k - proj_{u_1} v_k \\
u_k^{(2)} &= u_k^{(1)} - proj_{u_2} u_k^{(1)} \\
&\vdots \\
u_k^{(k-2)} &= u_k^{(k-3)} - proj_{u_{k-2}} u_k^{(k-3)} \\
u_k^{(k)} &= u_k^{(k-2)} - proj_{u_{k-1}} u_k^{(k-2)}
\end{aligned}$$

Pour trouver u_k, nous devons effectuer des calculs k-1.

2.2 Explication de cette méthode

La nouvelle méthode calcule chaque composante de u_k séparément, et bien qu'elles puissent également générer des résidus, les erreurs ne sont pas amplifiées.

3 application

3.1 tester les deux procédures sur les vecteur colonnes des matrices suivantes

$$\mathrm{A} = \begin{pmatrix} 1 & 2 & 3 \\ 2 & -1 & 2 \\ 1 & 1 & 2 \end{pmatrix} \qquad B = \begin{pmatrix} 1 & 1 & 1 \\ \varepsilon & 0 & 0 \\ 0 & \varepsilon & 0 \\ 0 & 0 & \varepsilon \end{pmatrix} \qquad \varepsilon = 10^{-10}$$

On obtient les résultats suivant :

$$\mathrm{A}_1 = \begin{pmatrix} 0.4082 & 0.7591 & 0.5071 \\ 0.8165 & -0.5521 & 0.1690 \\ 0.4082 & 0.3450 & -0.8452 \end{pmatrix} \qquad \mathrm{A}_2 = \begin{pmatrix} 0.4082 & 0.7591 & 0.5071 \\ 0.8165 & -0.5521 & 0.1690 \\ 0.4082 & 0.3450 & -0.8452 \end{pmatrix}$$

A_1 : le résultat de la version originale *A_2 : le résultat de la version modifiée*

$$B_1 = \begin{pmatrix} 1.0000 & 0 & 0 \\ 0.0000 & -0.7071 & -0.7071 \\ 0 & 0.7071 & 0 \\ 0 & 0 & 0.7071 \end{pmatrix} \qquad B_2 = \begin{pmatrix} 1.0000 & 0 & 0 \\ 0.0000 & -0.7071 & -0.4082 \\ 0 & 0.7071 & -0.4082 \\ 0 & 0 & 0.8165 \end{pmatrix}$$

B_1 : le résultat de la version originale *B_2 : le résultat de la version modifiée*

3.2 Comparer et interpréter les résultats obtenus par les deux algorithmes

On peut voir que les résultats obtenus par les deux algorithmes parfois sont différents. Pour une recherche approfondie, nous créons une matrice E (100 * 100) égale à X1 - X2 et calculons sa norme Frobenius (racine carrée de la somme carrée de tous les termes de matrice), que nous définissons comme le nombre d'erreurs entre les deux versions orthogonalisées de Schmitt.

On note

$$\| X \|_F = \sqrt{\sum_i \sum_j X_{i,j}^2}$$

On obtient deux nombres d'erreurs :

$$\mathrm{e\,(A)} = 5.9242 \times 10^{-31}$$

e(A) : le nombre d'erreurs de A

$$\mathrm{e\,(B)} = 0.2679$$

e(B) : le nombre d'erreurs de B

On peut voir que le nombre d'erreurs de A est zéro mais celui de B est grand.Pourquoi les résultats obtenus par les deux algorithmes parfois sont différents ? Et pourquoi les résultats obtenus par les deux algorithmes parfois ne sont pas différents ?

4 Le lien entre l'erreur et le nombre de conditions

```
c=zeros(100,1);
s=zeros(100,1);
for i=1:100
    a=rand(100);
    while(cond(a)<1000000)
       a=rand(100);
    end
    x1=Schmidt_orthogonalization_Original(a);
    x2=Schmidt_orthogonalization_Modified(a);
    s(i)=Erreur_moyenne(x1,x2);
    c(i)=cond(a);
end
plot(c,s,'--rs','LineWidth',2,...
                'MarkerEdgeColor','k',...
                'MarkerFaceColor','g',...
                'MarkerSize',10)
xlabel('condition');
ylabel(‘erreur');
```

Le but de cet algorithme est de montrer la relation entre l'erreur entre les deux versions d'orthogonalisation de Schmidt et le nombre de conditions dans un graphique linéaire.

Pourquoi ? Les deux versions sont mathématiquement correctes, mais les étapes de calcul sont différentes. Nous supposons que plus le nombre de conditions est élevé, plus le déterminant est proche de zéro et peut-être plus l'erreur entre les deux versions est élevé.

Il crée d'abord au hasard 100 matrices(100 * 100) dont le nombre de conditions est supérieur à 1 000 000. Un million, C'est nécessaire, nous avons besoin d'un plus grand nombre d'erreurs pour que le diagramme linéaire semble meilleur. Mais s'il est plus élevé, il faudra peut-être une journée entière pour produire un graphique.

Puis, à travers les deux fonctions de chaque version, nous obtenons une matrice X_1 (100 * 100) de la version originale (Schmidt_orthogonalization_original.m) et une autre matrice x_2 (100 * 100) de la version modifiée (Schmidt_orthogonalization_modified.m).

Ensuite, nous créons une matrice E (100 * 100) égale à X_1 - X_2 et calculons sa norme Frobenius (racine carrée de la somme carrée de tous les termes de matrice), que nous définissons comme le nombre d'erreurs entre les deux versions orthogonalisées de Schmidt. Avec 100 matrices, nous avons maintenant 100 nombres conditionnels et leurs erreurs associées.

Enfin, l'utilisez pour créer le graphique linéaire.L'axe horizontal est le nombre conditionnel, l'axe vertical est le nombre d'erreurs entre les deux versions orthogonalisées de Schmidt.

On note

$$\| X \|_F = \sqrt{\sum_i \sum_j X_{i,j}^2}$$

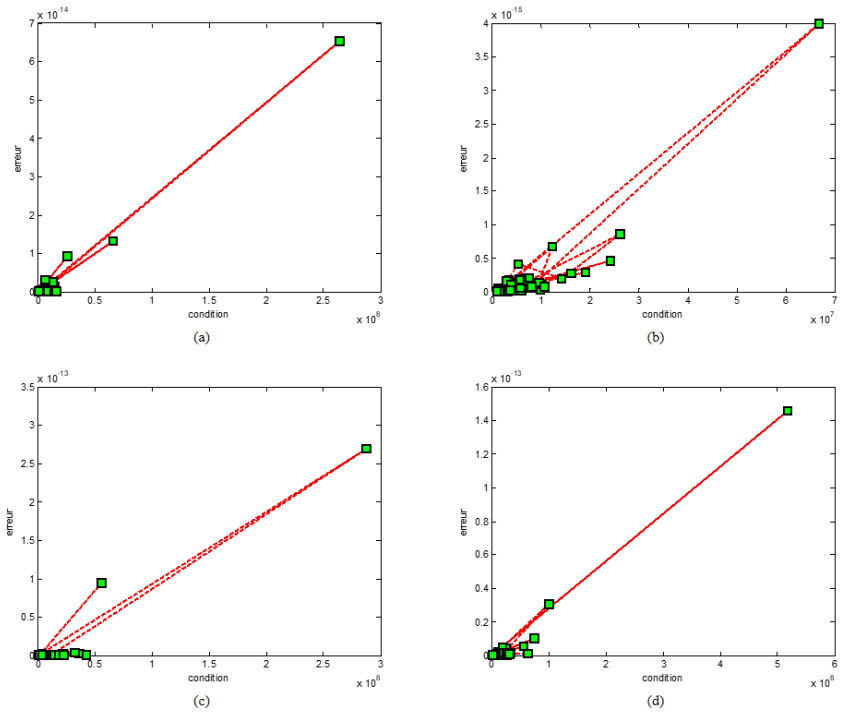


FIGURE 2 – *Le résultat de 4 exécutions du programme*

Nous avons exécuté le programme 4 fois, avec 4 diagrammes linéaires. (Figure 2) nous pouvons constater qu'il semble y avoir une certaine relation entre l'erreur entre les deux versions de l'orthogonalisation de Schmidt et le nombre de conditions. Bien que l'erreur soit très faible, nous pouvons trouver que, plus la valeur nominale de la condition, plus les deux versions de la valeur nominale. Pourquoi ?

En prenant la matrice (2 * 2) par exemple, nous avons maintenant deux vecteurs, colonnes A1 et A2. On dessine en coordonnées cartésiennes. Si cette matrice est de rang complet, A1

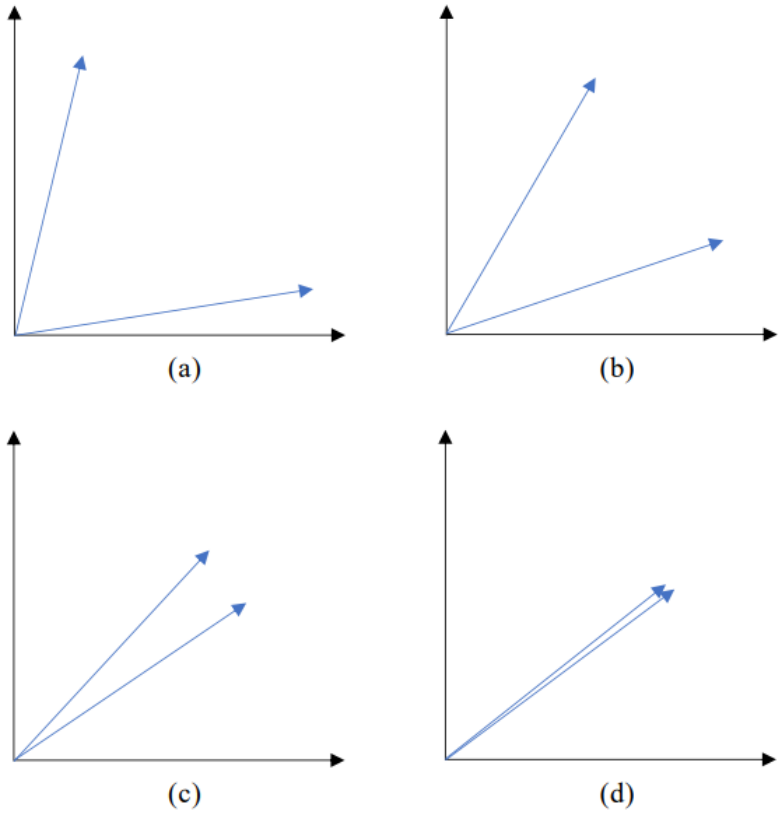


FIGURE 3 – *L'angle entre les deux vecteurs diminue progressivement*

et A2 doivent être libres, ce qui est nécessaire pour effectuer l'orthogonalisation Schmidt. En d'autres termes, les deux vecteurs ne peuvent pas être parallèles. Mais si nous tournons lentement l'un de ces vecteurs de sorte qu'ils soient presque parallèles. (Figure 3) l'ordinateur peut les considérer comme deux vecteurs parallèles, même s'ils ne sont pas strictement parallèles. Cela aggrave les problèmes causés par les résidus et conduit à une plus grande erreur entre les deux versions de l'orthogonalisation de Schmidt.

5 annexe

5.1 Schmidt_orthogonalization_Original.m

```
function b=Schmidt_orthogonalization_Original(a)
        [m,n] = size(a);
```

```
        b=zeros(m,n);
        b(:,1)=a(:,1);
        for i=2:n
            for j=1:i-1
                b(:,i)=b(:,i)-dot(a(:,i),b(:,j))/dot(b(:,j),b(:,j))*b(:,j);
            end
            b(:,i)=b(:,i)+a(:,i);
        end
        for k=1:n
        b(:,k)=b(:,k)/norm(b(:,k));
end
```

5.2 Schmidt_orthogonalization_Modified.m

```
function b=Schmidt_orthogonalization_Modified(a)
        [m,n] = size(a);
        b=zeros(m,n);
        b(:,1)=a(:,1);
        for i=2:n
            for j=1:i-1
                b(:,i)=a(:,i)-dot(a(:,i),b(:,j))/dot(b(:,j),b(:,j))*b(:,j);
                a(:,i)=b(:,i);
            end
        end
        for k=1:n
        b(:,k)=b(:,k)/norm(b(:,k));
end
```

5.3 erreur_moyenne.m

```
function s=Erreur_moyenne(a,b)
        d=a-b;
        s=0;
        [m,n] = size(a);
```

```
        for i=1:m
                    for j=1:n
                s=s+d(i,j)^2;
            end
        end
end
```